

Tarea 04 y 05: Sistema Multi-Agente con RAG, A2A, MCP, Memoria

Curso de Inteligencia Artificial
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica

I. OBJETIVO

El propósito de esta tarea es diseñar, implementar y evaluar un sistema multi-agente general con la capacidad de responder preguntas a partir de los apuntes generados por los estudiantes del curso de Inteligencia Artificial.

El sistema deberá combinar técnicas de *Retrieval-Augmented Generation* (RAG), orquestación entre agentes, memoria conversacional, uso controlado de herramientas externas y observabilidad. Además, deberá incluir un componente de datos transaccionales ficticios para simular el uso de información sensible a través de un servidor MCP.

El objetivo principal es que el estudiante comprenda cómo construir una aplicación moderna basada en agentes, donde un modelo de lenguaje no solamente responde preguntas, sino que también decide qué agente consultar, qué herramienta utilizar, cuándo recuperar información, cuándo consultar memoria histórica y cómo justificar sus acciones.

II. DESCRIPCIÓN GENERAL

Cada grupo deberá construir un sistema compuesto por los siguientes elementos:

- Una base documental formada por los apuntes del curso, que utilizando un sistema RAG pueda consultar información relevante dichos apuntes.
- Un conjunto de agentes especializados coordinados por un agente orquestador.
- Un mecanismo de comunicación entre agentes inspirado en A2A.
- Un MCP Server que permita consultar una base de datos relacional con transacciones ficticias.
- Una memoria conversacional temporal e histórica.
- Observabilidad mediante Langfuse o una herramienta equivalente.
- Una interfaz web para interactuar con el sistema.

III. PASOS A SEGUIR / INSTRUCCIONES

III-A. Base documental de apuntes

La base documental estará formada por los apuntes realizados por los estudiantes del curso de Inteligencia Artificial. Los apuntes compartidos con el grupo y utilizados como fuente principal de conocimiento académico para el sistema RAG.

La información debe ser extraída de cada documento y acompañada de metadatos que permitan mejorar la recuperación de información. Algunos ejemplos de metadatos son:

- Nombre del archivo.
- Autor o autores del apunte.
- Fecha de elaboración.
- Tema principal.
- Semana o clase asociada.
- Sección o subsección del documento.

III-B. Preprocesamiento textual

Cada grupo deberá aplicar un proceso de limpieza y preparación de los documentos. Como mínimo, se debe realizar lo siguiente:

- Extraer el contenido textual de los documentos.
- Limpiar caracteres problemáticos.
- Normalizar tildes, mayúsculas y minúsculas cuando sea necesario.
- Eliminar contenido irrelevante o ruido del documento.
- Aplicar estrategias segmentación de texto.

III-C. Tokenización, embeddings y base vectorial

Una vez preprocesado el texto, los fragmentos deberán transformarse en embeddings contextuales para ser almacenados en una base de datos vectorial.

Pueden utilizar el modelo de embeddings de su preferencia. Se recomienda justificar la selección del modelo considerando costo, disponibilidad, idioma, tamaño del embedding y calidad esperada.

El grupo deberá implementar dos configuraciones RAG distintas, cambiando al menos la estrategia de segmentación. Ambas configuraciones deberán ser evaluadas y comparadas en el informe.

III-D. Arquitectura multi-agente

El sistema deberá estar compuesto por varios agentes especializados según se muestra en la Tabla I

III-E. Comunicación entre agentes

El sistema deberá implementar una forma clara de comunicación entre agentes. Se recomienda utilizar un enfoque compatible con A2A o, como mínimo, definir contratos formales para cada agente.

Cada agente deberá tener una definición que incluya:

- Nombre del agente.
- Descripción.
- Rol dentro del sistema.
- Herramientas que puede utilizar.

Cuadro I
AGENTES MÍNIMOS DEL SISTEMA.

Agente	Responsabilidad	Modelo sugerido
Agente Orquestador	Recibe la pregunta del usuario, decide qué agente o herramienta utilizar, coordina el flujo de trabajo y construye la respuesta final. Debe ser el agente más riguroso del sistema.	Modelo más fuerte disponible
Agente RAG	Consulta la base vectorial de apuntes, recupera fragmentos relevantes y genera respuestas basadas en documentos del curso.	Modelo medio
Agente Web Search	Realiza búsquedas en internet únicamente cuando el usuario lo solicita explícitamente o cuando el orquestador justifica que se requiere información externa.	Modelo medio
Agente Resumidor	Resume conversaciones, resultados de búsqueda, respuestas largas o historial de una sesión. Puede utilizar un modelo más pequeño para reducir costos.	Modelo económico
Agente Transaccional	Consulta la base de datos ficticia mediante el MCP Server. No debe acceder directamente a la base de datos.	Modelo medio

- Entradas esperadas.
- Salidas esperadas.
- Restricciones.
- Ejemplo de llamada.

Ejemplo de definición mínima:

```
{
  "agent_name": "NotesRAGAgent",
  "description": "Agente especializado en
    recuperar informacion desde los apuntes
    del curso.",
  "skills": [
    "search_notes",
    "answer_from_notes"
  ],
  "allowed_tools": [
    "rag_tool"
  ],
  "restrictions": [
    "Debe citar documentos y autores.",
    "No debe inventar fuentes.",
    "No debe usar busqueda web."
  ]
}
```

III-F. Herramientas del sistema

El sistema deberá contar con herramientas especializadas. Como mínimo:

- **RAG Tool:** recupera información desde la base vectorial de apuntes.
- **WebSearch Tool:** realiza búsqueda web cuando sea permitido.
- **Memory Tool:** consulta y actualiza la memoria conversacional.
- **MCP Transaction Tool:** consulta datos transaccionales ficticios mediante MCP.

III-G. MCP Server y base de datos transaccional ficticia

Cada grupo deberá construir un MCP Server que exponga herramientas controladas para consultar una base de datos relacional ficticia. La base de datos deberá ejecutarse mediante Docker y deberá ser poblada utilizando un script de Python o archivos SQL.

La base de datos debe simular información medianamente crítica, por ejemplo transacciones financieras ficticias de una organización. No se deben utilizar datos reales.

El MCP Server no debe exponer una herramienta genérica de ejecución SQL libre. Es decir, no se permite una herramienta como:

```
execute_sql(query)
```

En su lugar, se deben exponer herramientas controladas como:

- `get_transaction_by_id(transaction_id)`
- `search_transactions(filters)`
- `get_customer_risk_summary(customer_id)`
- `get_recent_flagged_transactions(days)`
- `create_fraud_case(transaction_id, reason, severity)`

III-H. Base de datos ficticia

La base de datos debe contener, como mínimo, las siguientes entidades:

- Clientes ficticios.
- Cuentas ficticias.
- Transacciones ficticias.
- Casos de revisión o fraude ficticio.
- Reglas de uso de herramientas.

La base debe incluir transacciones normales y transacciones sospechosas. Algunos ejemplos de comportamiento sospechoso son:

- Montos inusualmente altos.
- Muchas transacciones en un periodo corto.
- Transacciones en países distintos durante el mismo día.
- Transacciones realizadas de madrugada.
- Transacciones fallidas seguidas por transacciones aprobadas.
- Actividad inusual para el nivel de riesgo del cliente.

III-I. Reglas de uso del MCP Server

Cada grupo deberá definir reglas de uso seguro para el MCP Server. Como mínimo, deben considerar las siguientes:

- Toda llamada al MCP debe incluir una justificación.

- No se deben mostrar números de cuenta completos. Solo se permiten los últimos cuatro dígitos.
- No se deben permitir consultas masivas sin filtros.
- No se deben modificar transacciones existentes.
- Las consultas con información sensible deben ser anonimizadas o rechazadas.
- Las búsquedas históricas deben limitarse por rango de fechas.
- El agente debe explicar qué datos utilizó para llegar a una conclusión.

III-J. Memoria conversacional e histórica

El sistema deberá implementar dos tipos de memoria:

- **Memoria temporal de sesión:** mantiene coherencia entre preguntas consecutivas dentro de la misma conversación.
- **Memoria histórica persistente:** permite consultar preguntas anteriores, respuestas previas y resúmenes de sesiones pasadas.

La memoria histórica debe almacenarse en la base de datos o en un sistema persistente equivalente. Debe permitir que el agente responda preguntas como:

- ¿Qué pregunté anteriormente sobre descenso del gradiente?
- Resume las preguntas realizadas en esta sesión.
- Compara esta pregunta con una pregunta anterior.
- ¿Ya habíamos consultado transacciones sospechosas hoy?

III-K. Observabilidad con Langfuse

El sistema deberá registrar trazas de ejecución utilizando Langfuse como lo muestra la Figuras 1 y 2 o una herramienta equivalente. La observabilidad debe permitir inspeccionar:

- Entrada del usuario.
- Agente que recibió la solicitud.
- Herramientas utilizadas.
- Fragmentos recuperados por el RAG.
- Llamadas al MCP Server.
- Llamadas al modelo de lenguaje.
- Latencia.
- Errores.
- Costo aproximado, si la herramienta lo permite.

Se permite utilizar Langfuse Cloud o una instalación local mediante Docker.

III-L. Aplicación web

El sistema debe ejecutarse mediante una interfaz web donde sea posible interactuar con el agente. Se permite utilizar herramientas como Streamlit, Gradio, FastAPI con frontend simple, Next.js u otra herramienta equivalente.

La aplicación debe permitir demostrar:

- Preguntas sobre los apuntes.
- Preguntas que requieran memoria conversacional.
- Preguntas que activen Web Search.
- Preguntas que activen el MCP Server.

IV. EVALUACIÓN EXPERIMENTAL

Cada grupo deberá construir un pequeño conjunto de pruebas para evaluar el sistema. Como mínimo:

- 10 preguntas factuales sobre los apuntes.
- 5 preguntas de comparación entre temas.
- 5 preguntas conversacionales de seguimiento.
- 5 preguntas fuera del alcance de los apuntes.
- 5 preguntas que requieran búsqueda web explícita.
- 5 preguntas sobre transacciones ficticias.

V. DEMOSTRACIÓN PRESENCIAL

Durante la demostración, cada grupo deberá mostrar:

1. Preguntas respondidas desde los apuntes.
2. Pregunta de seguimiento usando memoria temporal.
3. Preguntas que consulte memoria histórica.
4. Preguntas que use Web Search.
5. Preguntas que consulte la base transaccional mediante MCP.
6. Preguntas que intente acceder a información sensible y sea rechazada o anonimizada.
7. Evidencia de trazas en Langfuse.
8. Evidencia de registros en la base de datos.

VI. PLAN DE ENTREGAS Y EVALUACIÓN POR ETAPAS

El proyecto se entregará en dos partes. La primera entrega corresponde a un prototipo funcional del sistema, mientras que la segunda entrega corresponde a la versión final completa, incorporando las observaciones realizadas durante la revisión de la primera parte.

VI-A. Primera entrega: prototipo funcional

En la primera entrega, cada grupo deberá presentar una versión inicial funcional del sistema. Esta entrega tiene como objetivo validar que el grupo cuenta con una arquitectura base operativa antes de incorporar los componentes más avanzados del proyecto.

Como mínimo, la primera entrega deberá incluir:

- Un **Agente Orquestador** capaz de recibir la pregunta del usuario y decidir el flujo básico de ejecución.
- Un **Agente RAG** o herramienta RAG capaz de consultar los apuntes del curso.
- Una base vectorial funcional con documentos procesados y fragmentados con una respuesta generada a partir de fragmentos recuperados desde los apuntes.
- Evidencia de **observabilidad** utilizando Langfuse o una herramienta equivalente.
- Registro de al menos una traza completa que incluya la pregunta del usuario, la llamada al modelo, la recuperación RAG y la respuesta generada.
- Una interfaz mínima para probar el sistema.
- Un avance del informe en formato IEEE que documente la arquitectura inicial, decisiones tomadas y problemas encontrados.

En esta primera etapa no es obligatorio implementar todavía el MCP Server, la base de datos transaccional, el Agente Web

Filters	Timestamp	Name	Input	Output
Environment	2026-05-26 14:41:11	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Mode: SELECT TEXT	2026-05-26 14:20:00	assistant-tool-results		("messages": [{"content": "[context] Today is 2026-05-26. ¿Cuánta...
default	2026-05-26 14:19:58	assistant-question	"[context] Today is 2026-05-26. ¿Cuántas transacciones anuladas t...	("messages": [{"content": "[context] Today is 2026-05-26. ¿Cuánta...
See docs on how to add environments to your traces.	2026-05-26 14:14:59	assistant-tool-results		("messages": [{"content": "[context] Today is 2026-05-26. ¿Cuánto ...
Trace Name	2026-05-26 14:14:57	assistant-question	"[context] Today is 2026-05-26. ¿Cuánto vendí ayer?"	("messages": [{"content": "[context] Today is 2026-05-26. ¿Cuánto ...
Mode: SELECT TEXT	2026-05-26 12:00:53	assistant-question	"[context] Today is 2026-05-26. Por hora del día, cuántas transacci...	("messages": [{"content": "[context] Today is 2026-05-26. Por hora...
assistant-question 24	2026-05-26 12:00:36	assistant-question	"[context] Today is 2026-05-26. Cuál es el promedio del monto de l...	("messages": [{"content": "[context] Today is 2026-05-26. Cuál es ...
assistant-tool-results 2	2026-05-26 11:37:34	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Trace ID	2026-05-26 11:37:26	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
User ID	2026-05-26 11:37:19	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Session ID	2026-05-26 11:37:07	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Metadata	2026-05-26 11:37:05	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Version	2026-05-26 11:37:02	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Release	2026-05-26 11:36:58	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Bookmarked	2026-05-26 11:36:55	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Comment Count	2026-05-26 11:36:52	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Comment Content	2026-05-26 11:36:49	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Tags	2026-05-26 11:36:47	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Level	2026-05-26 11:36:44	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Latency	2026-05-26 11:36:42	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
Input Tokens	2026-05-26 11:36:39	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
	2026-05-26 11:36:36	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...
	2026-05-26 11:36:33	assistant-question	[{"type": "text", "text": "[context] Today is 2026-05-26 in the terminal'...	("messages": [{"content": [{"type": "text", "text": "[context] Today is ...

Figura 1. Trazas registradas en Langfuse durante la ejecución del sistema multi-agente.

Search, el Agente Resumidor, el Agente Transaccional ni la memoria histórica persistente. Sin embargo, el grupo sí deberá explicar en el informe preliminar cómo planea incorporar estos componentes en la segunda entrega.

VI-B. Evaluación de la primera entrega

La primera entrega será evaluada sobre 30 puntos sobre la rubrica descrita en la Tabla II. Esta evaluación permitirá brindar retroalimentación temprana al grupo y detectar problemas de arquitectura, recuperación de información, trazabilidad u organización del proyecto.

Nota importante: las observaciones realizadas durante la revisión de la primera entrega deberán ser atendidas obligatoriamente en la segunda entrega. La segunda entrega no debe verse como un proyecto separado, sino como una evolución directa del prototipo inicial. Si el grupo no corrige problemas señalados en la primera revisión, esto podrá afectar la calificación de los criterios relacionados en la entrega final.

VI-C. Segunda entrega: sistema completo

La segunda entrega corresponde a la versión final del proyecto. En esta etapa, el grupo deberá mejorar el prototipo presentado en la primera entrega e incorporar los componentes restantes del sistema multi-agente.

Como mínimo, la segunda entrega deberá incluir:

- Corrección de las observaciones realizadas en la primera entrega.

- Mejora del Agente Orquestador.
- Implementación del Agente Web Search.
- Implementación del Agente Resumidor utilizando un modelo más pequeño o económico.
- Implementación del Agente Transaccional.
- Implementación del MCP Server para consultar la base de datos transaccional ficticia.
- Base de datos relacional en Docker con datos ficticios generados mediante SQL o script de Python.
- Reglas de seguridad para el uso del MCP Server.
- Registro de llamadas realizadas por agentes y herramientas.
- Memoria temporal de sesión.
- Memoria histórica persistente.
- Evidencia completa de observabilidad en Langfuse o herramienta equivalente.
- Evaluación experimental del sistema.
- Informe final en formato IEEE.

VI-D. Evaluación de la segunda entrega

La segunda entrega será evaluada sobre 70 puntos. Esta evaluación considerará tanto la implementación de los componentes restantes como la mejora del sistema a partir de la retroalimentación recibida en la primera entrega sobre la rubrica descrita en la Tabla III.

Nota sobre trazabilidad: La trazabilidad del sistema deberá evidenciarse mediante registros de llamadas, logs, trazas en

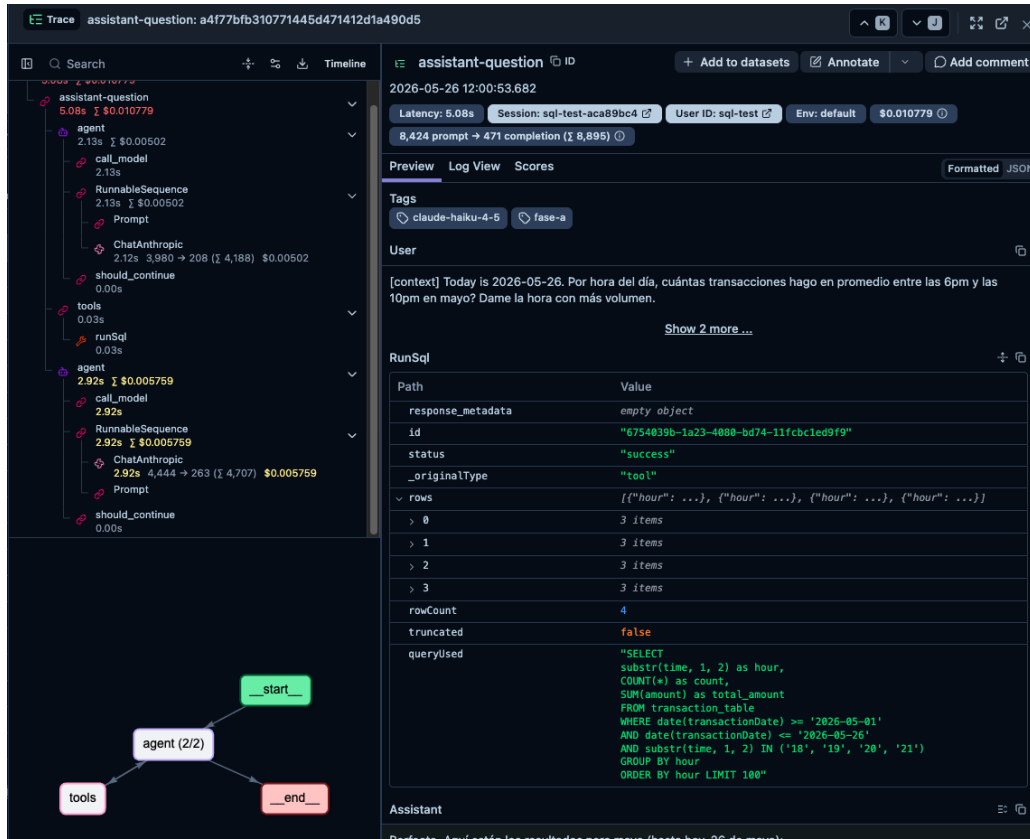


Figura 2. Trazas registradas en Langfuse a detalle

Cuadro II
EVALUACIÓN DE LA PRIMERA ENTREGA.

Criterio	Descripción	Puntos
Agente Orquestador inicial	El sistema cuenta con un orquestador funcional que recibe preguntas, coordina el flujo básico y decide cuándo utilizar el RAG.	7 pts
RAG sobre apuntes	El sistema procesa documentos, genera embeddings, almacena fragmentos en una base vectorial y recupera información relevante desde los apuntes.	9 pts
Respuesta fundamentada	Las respuestas utilizan fragmentos recuperados desde los apuntes y muestran evidencia de que el sistema no responde únicamente desde conocimiento general del modelo.	4 pts
Observabilidad	El sistema registra trazas en Langfuse o herramienta equivalente, incluyendo entrada del usuario, recuperación RAG, llamada al modelo y respuesta generada.	5 pts
Interfaz y demostración	El grupo presenta una interfaz mínima funcional y demuestra el flujo completo de una consulta.	3 pts
Informe preliminar	El informe documenta la arquitectura inicial, decisiones técnicas, problemas encontrados y plan para la segunda entrega.	2 pts
Total		30 pts

Langfuse y registros almacenados en la base de datos cuando corresponda. El objetivo es que el grupo pueda explicar qué agentes y herramientas participaron en cada consulta, por qué fueron utilizados y qué resultado produjeron.

VI-E. Calificación final del proyecto

La calificación final del proyecto se calculará de la siguiente manera:

- **Primera entrega:** 30 puntos.
- **Segunda entrega:** 70 puntos.
- **Total:** 100 puntos.

La segunda entrega deberá incluir una sección en el informe titulada *Correcciones realizadas a partir de la primera entrega*, donde el grupo explique qué observaciones recibió, cuáles fueron corregidas y cómo se evidencian dichas mejoras en la versión final del sistema.

VII. ENTREGA

El informe deberá realizarse en $\text{\LaTeX}$ (Overleaf) utilizando la plantilla IEEE para artículos científicos. Además, se debe adjuntar el código fuente del sistema implementado.

Cuadro III
EVALUACIÓN DE LA SEGUNDA ENTREGA.

Criterio	Descripción	Puntos
Mejoras sobre la primera entrega	El grupo corrige las observaciones recibidas, mejora la arquitectura inicial y demuestra evolución del prototipo.	8 pts
RAG completo y comparación experimental	Se implementan al menos dos estrategias de segmentación y se comparan sus resultados mediante preguntas de prueba.	8 pts
Arquitectura multi-agente	El sistema incorpora agentes especializados, incluyendo orquestador, RAG, Web Search, resumidor y transaccional.	10 pts
Orquestación y comunicación entre agentes	El orquestador decide correctamente qué agente o herramienta utilizar. Se definen contratos formales o un enfoque compatible con A2A.	7 pts
MCP Server y base transaccional	Se implementa un MCP Server con herramientas controladas para consultar una base de datos relacional ficticia ejecutada en Docker.	10 pts
Reglas de seguridad y trazabilidad	El sistema aplica reglas de acceso, evita consultas masivas sin justificación, anonimiza datos sensibles y registra las llamadas realizadas por agentes y herramientas.	7 pts
Memoria temporal e histórica	El sistema mantiene contexto dentro de una sesión y permite consultar información de interacciones anteriores.	6 pts
Observabilidad completa	Se registran trazas completas que incluyen llamadas a agentes, herramientas, RAG, MCP, modelo de lenguaje, latencia y errores.	6 pts
Evaluación experimental	El grupo presenta un conjunto de preguntas de prueba, resultados, análisis de errores y discusión de limitaciones.	4 pts
Informe final y reproducibilidad	El informe IEEE es claro, completo y contiene instrucciones para ejecutar el sistema, diagramas, evidencias y conclusiones.	4 pts
Total		70 pts

La entrega final consistirá en un archivo comprimido (.zip) que contenga:

- Código fuente en L^AT_EX.
- PDF del informe.
- Código fuente de la aplicación.
- Código de los agentes.
- Código del MCP Server.
- Archivos SQL o script de Python para poblar la base de datos.
- Capturas o evidencia de Langfuse.

VIII. ESTRUCTURA MÍNIMA DEL INFORME

El informe debe incluir, como mínimo:

- Introducción.
- Descripción general de la arquitectura.
- Diseño de agentes y responsabilidades.
- Diseño del RAG y comparación de segmentación.
- Diseño del MCP Server.
- Reglas de seguridad y acceso.
- Memoria temporal e histórica.
- Observabilidad.
- Evaluación experimental.
- Análisis de errores.
- Conclusiones.